

PART 1

Problem 1: Synchronization (Lost Update Anomaly)

The synchronization issue occurs in the **read-modify-write** pattern in the database interaction layer of the FastAPI backend. The API fetches a document (e.g., GET /documents/{id}), modifies it in memory, then saves it back (PUT). When two users read the same version nearly simultaneously, both updates are applied independently. The last write silently overwrites the first because there is no optimistic/pessimistic locking or version checking. This classic Lost Update happens due to missing concurrency control in both the SQLAlchemy models and API endpoints.

Problem 2: Coordination (Dropped Clerk Webhooks)

Clerk webhooks are handled by a simple synchronous endpoint that directly updates the local user subscription record. If a network blip drops the webhook (or the server is down during delivery), the event is lost forever. There is no idempotency checking, retry mechanism, or reconciliation with Clerk's API. This causes permanent state inconsistency — Clerk shows non-premium while StudySync still grants premium access.

Problem 3: Fault Tolerance (Blocking LLM Calls)

LLM API calls are made **synchronously** inside request handlers. When the external service slows down or times out (up to 60s), the FastAPI/Uvicorn worker remains blocked. Under load, all workers get exhausted, causing the entire application to hang for all users. The external dependency becomes a single point of failure due to tight coupling and lack of timeouts, async handling, or resilience patterns.

Part 2

1. Synchronization Solution — Optimistic Locking

StudySync should use **Optimistic Locking** with document version numbers.

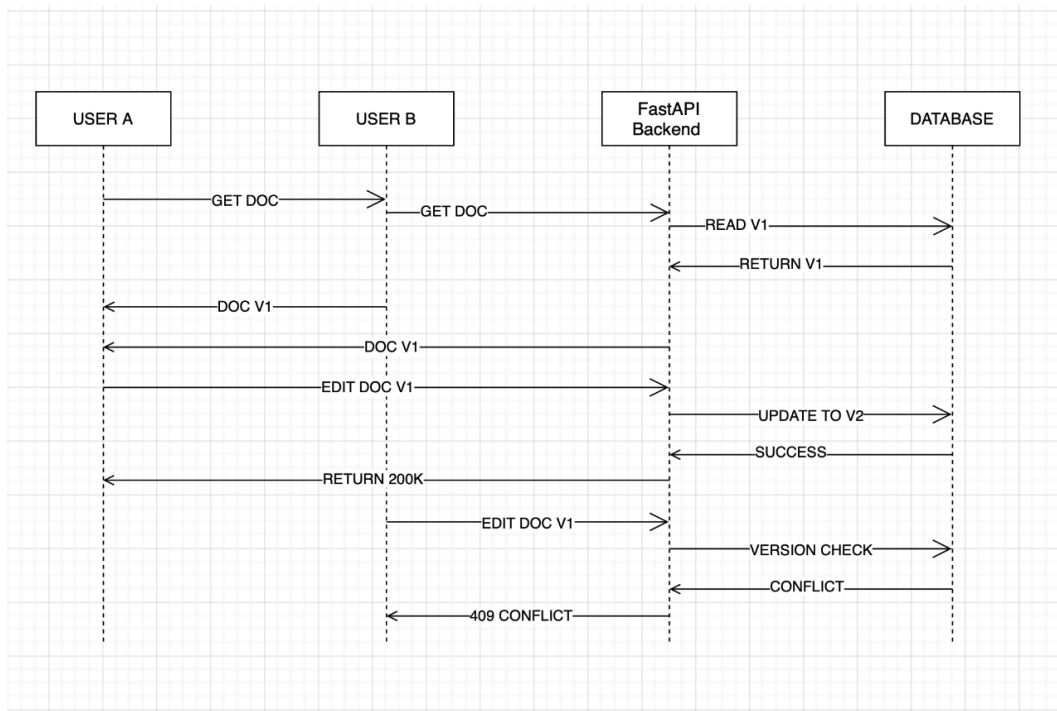
Each document stores:

- **document_id, content, version**

Process:

- Client fetches document with its current version.
- On update, client sends new content + original version.
- Backend only updates if versions match and then increments the version.
- On mismatch, return **HTTP 409 Conflict** with the latest version so the client can re-fetch and retry.

This prevents silent overwrites.



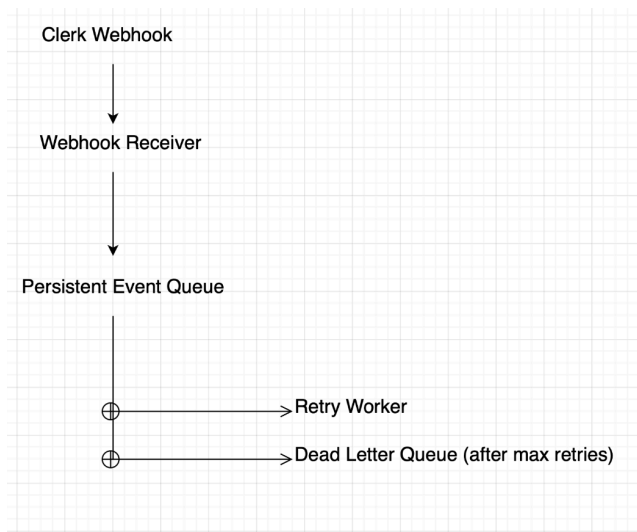
2. Coordination Solution — Fault-Tolerant Webhook Handling

Webhook handling should use:

- persistent event storage, retries, idempotency keys, and a Dead-Letter Queue (DLQ).

Improved Flow

1. Use Clerk's unique `event_id` as **idempotency key**.
2. Store received events in a `processed_events` table (with unique constraint) before processing.
3. Process events asynchronously using Celery/Redis queue with automatic retries.
4. Implement a daily reconciliation job that queries Clerk API to fix any drifted subscription states.
5. Send permanently failed events to a Dead-Letter Queue.

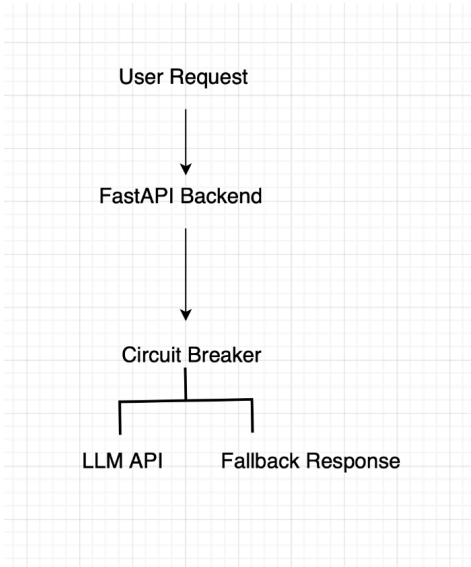


This ensures cancellations are never permanently lost.

3. Fault Tolerance Solution — Circuit Breaker Pattern

Wrap LLM calls with a Circuit Breaker (pybreaker library) + `httpx.AsyncClient`:

- **Closed:** Normal calls with timeout (5s) and retries.
- **Open:** Fast-fail and return fallback response after 5 failures.
- **Half-Open:** Periodic test requests. Fallback: “AI assistant is temporarily unavailable. Please try again later.”



Additional Improvements

- asynchronous requests, short timeouts, background workers.

4. CAP Theorem Trade-offs

Component	Priority	Trade-off
Optimistic Locking	Consistency	Some updates may be rejected
Webhook Retries	Availability + Partition Tolerance	Temporary inconsistency accepted
Circuit Breaker	Availability + Low Latency	Fallback responses may be less accurate

- **Optimistic Locking:** Prioritizes **Consistency** (prevents lost updates) at slight cost to **Availability** (rare 409 retries).
- **Webhook + Reconciliation:** Favors **Availability** and **Partition Tolerance** using **Eventual Consistency**.
- **Circuit Breaker:** Strongly favors **Availability** and low **Latency** over strong Consistency during outages.

Overall, the system leans AP (high availability) while enforcing strong consistency only where critical (document edits). This is suitable for an ed-tech platform.